Davante

14

BASE DE DATOS

Transacciones y problemas de concurrencia

ÍNDICE

/ 1. Introducción y contextualización práctica 4

/ 2. Transacciones 5

2.1. Propiedades de las transacciones 5

2.2. Control de transacciones 5

2.3. Estados de las transacciones 6

2.4. Pasos para realizar una transacción. START TRANSACTION, COMMIT y ROLLBACK 6

/ 3. SAVEPOINT, ROLLBACK TO SAVEPOINT y RELEASE SAVEPOINT, y estado de datos durante la transacción 7

3.1. SAVEPOINT 8

3.2. ROLLBACK TO SAVEPOINT 8

3.3. RELEASE SAVEPOINT 8

3.4. Estado de los datos durante las transacciones 8

/ 4. Problemas de concurrencia de transacciones 8

/ 5. Control de concurrencia: Técnicas de bloqueo pesimistas y optimistas 10

/ 6. Transacciones distribuidas 11

6.1. Requerimientos de transacciones distribuidas 11

6.2. Recuperación de transacciones distribuidas 11

6.3. COMMIT de dos fases 12

© MEDAC ISBN: 978-84-19885-68-5
Reservados todos los derechos. Queda rigurosamente prohibida, sin la autorización escrita de los titulares del copyright, bajo las sanciones establecidas en las leyes, la reproducción, transmisión y distribución total o parcial de esta obra por cualquier medio o procedimiento, incluidos la reprografía y el tratamiento informático.

ÍNDICE

/ 7. Políticas de bloqueo 12

7.1. Niveles de bloqueo 12

7.2. Tipos de bloqueos 13

/ 8. Niveles de procesamiento de consulta: Procesadores locales y procesadores distribuidos 14

8.1. Procesadores locales 14

8.2. Procesadores distribuidos 14

/ 9. Caso práctico 1: “Manejo de transacciones” 15

/ 10. Caso práctico 2: “Padres e hijos” 16

/ 11. Resumen y resolución del caso práctico de la unidad 16

/ 12. Bibliografía 18

OBJETIVOS

Comprender el concepto de transacción, incluyendo sus propiedades, estados, comandos y mecanismos de control.

Conocer las transacciones distribuidas y sus niveles de procesamiento.

Entender el concepto de concurrencia, los problemas que puede causar y las técnicas para controlarla.

Examinar las diversas políticas y tipos de bloqueo disponibles en los sistemas de bases de datos.

/ 1. Introducción y contextualización práctica

A menudo, puede ser útil realizar ciertas acciones sobre una tabla de forma conjunta. Estas acciones pueden incluir modificaciones, eliminación de datos o la inserción de nueva información. Son operaciones que pueden ejecutarse mediante transacciones, que agrupan todas estas tareas relacionadas como una sola unidad. En este tema, exploraremos en detalle este concepto y las diferentes formas de implementar transacciones.



Fig.1. Podemos practicar transacciones en Oracle SQL Developer aplicando la teoría del tema.

Además, hoy en día, una base de datos es utilizada por organizaciones completas, compuestas por varios usuarios con distintos perfiles, como hemos visto en temas anteriores. Estos usuarios acceden a la misma información, lo que implica que los datos pueden ser modificados por múltiples personas al mismo tiempo. En este apartado, examinaremos los mecanismos que se implementan para gestionar esta situación, garantizando siempre la consistencia e integridad de los datos almacenados en la base de datos.

Finalmente, se abordará el concepto y análisis de las políticas de bloqueo que existen en las bases de datos, enfocándonos particularmente en el caso de Oracle. Estas políticas de bloqueo son fundamentales para gestionar la concurrencia de manera efectiva.

A continuación, vamos a plantear un caso práctico a través del cual podremos aproximarnos de forma práctica a la teoría de este tema.

Escucha el siguiente audio donde planteamos la contextualización práctica de este tema. Encontrarás su resolución en el apartado 'Resumen y resolución del caso práctico'.

Audio intro. "Solucionando problemas de concurrencia"

https://on.soundcloud.com/2BgCAKFcxcnWVNNn9

/ 2. Transacciones

Una transacción se define como un **conjunto de instrucciones SQL** de tipo DML (Lenguaje de Modificación de Datos) que **se ejecutan como una sola operación**, permitiendo que todas esas instrucciones **se apliquen simultáneamente**. En otras palabras, las transacciones agrupan estas instrucciones para que se ejecuten de manera indivisible y unitaria.

El uso de transacciones garantiza que diversas operaciones se realicen de **manera segura** y que los datos puedan **recuperarse** en caso de que ocurra algún error durante la ejecución. Sin embargo, esto puede **aumentar el tiempo de ejecución** de las operaciones.

2.1. Propiedades de las transacciones

Las propiedades conocidas como **ACID** (*Atomicity, Consistency, Isolation y Durability*) aseguran que las transacciones se realicen de manera segura en la base de datos. Un SGBD es **compatible con ACID (*ACID compliant*)** cuando permite la ejecución de transacciones.

En la siguiente tabla, se muestran estas cuatro propiedades:

Propiedad	Descripción
Atomicidad (A)	Si alguna de las operaciones dentro de una transacción falla, ninguna de las operaciones se aplica. Por ejemplo, si una transacción modifica 80 registros, pero falla al modificar los últimos 10, el sistema revierte los cambios de los primeros 70 registros. La transacción no se puede dividir, debe ejecutarse en su totalidad o no ejecutarse en absoluto.
Consistencia (C)	La transacción debe iniciar y finalizar manteniendo la integridad y consistencia de los datos en el sistema, independientemente de si se completa o no. Esta propiedad asegura que la base de datos permanezca en un estado estable y consistente después de la transacción.
Aislamiento (I)	Los resultados de una transacción no deben ser visibles para otras transacciones hasta que la primera haya terminado. Esto evita que las transacciones concurrentes interfieran entre sí. Cada transacción opera de manera aislada, y si dos transacciones requieren acceso a los mismos datos, una ha de esperar a que la otra termine.
Durabilidad (D)	Los cambios resultantes de transacciones completadas con éxito deben ser permanentes. Esto significa que, una vez realizados, los cambios en la base de datos no se perderán.

Tabla 1. Propiedades ACID de las transacciones.

2.2. Control de transacciones

El control de transacciones engloba las estrategias y los métodos aplicados para **controlar** la **ejecución de las transacciones**, **garantizando que se respeten las propiedades ACID**. Incluye:

- **Control de concurrencia**: Procedimientos que evitan que transacciones que se ejecutan simultáneamente interfieran unas con otras, previniendo así posibles inconsistencias en la base de datos.
- **Gestión de bloqueos**: Implementación de bloqueos para impedir que diferentes transacciones accedan de manera simultánea a los mismos datos.
- **Recuperación de transacciones**: Conjunto de técnicas diseñadas para restaurar la base de datos a un estado consistente tras un fallo, asegurando que las transacciones finalizadas permanezcan intactas y que las no completadas no comprometan la integridad de los datos.

2.3. Estados de las transacciones

El ciclo de vida de una transacción se compone de varios estados, que se describen a continuación.

- **Inicio (*start*)**: Es el momento en que se inicia una transacción. Aquí, una operación o una serie de operaciones se consideran como una única unidad de trabajo.
- **En progreso (*active*)**: La transacción está en proceso, y se están ejecutando las operaciones correspondientes.
- **Completada (*committed*)**: La transacción ha concluido con éxito. Todas las operaciones realizadas se confirman y se guardan de forma permanente en la base de datos.
- **Fallida (*failed*)**: Una transacción entra en este estado si se produce un error durante su ejecución. En esta situación, la transacción no puede proceder hacia una finalización exitosa y debe ser revertida.
- **Abortada (*aborted* o *rolled back*)**: Después de que una transacción es considerada 'fallida', entra en el estado de 'abortada'. En este punto, todos los cambios efectuados durante la transacción son deshechos, devolviendo la base de datos a su estado previo al inicio de la transacción.
- **En espera o parcialmente comprometida (*partially committed* o *prepared*)**: En sistemas de bases de datos distribuidos, una transacción puede quedar en espera hasta que todas las partes del sistema confirmen que están listas para proceder con la confirmación final (*commit*).

Vídeo 1 "Ejemplos de uso de transacciones en Oracle 18c"
https://bit.ly/34GAaKw

2.4. Pasos para realizar una transacción. START TRANSACTION, COMMIT y ROLLBACK

Una transacción suele iniciarse con la primera instrucción DML que se ejecuta y se concluye con una instrucción **COMMIT** (si se decide confirmar la transacción) o con una instrucción **ROLLBACK** (si se decide cancelarla):

- **ROLLBACK**: Revierte todos los cambios realizados por la transacción, devolviendo la base de datos al estado anterior al inicio de la transacción. Esta acción se lleva a cabo automáticamente si ocurre un problema de sesión, una caída del sistema o un fallo en la comunicación.
- **COMMIT**: Confirma todas las instrucciones incluidas en la transacción, haciendo que los cambios sean permanentes. Es importante utilizar COMMIT solo cuando se está seguro de que los cambios son correctos y necesarios. El cierre exitoso de una sesión provoca un COMMIT implícito.

Los **pasos para ejecutar una transacción son** los siguientes:

1. **Iniciar la transacción** con sentencias como **START TRANSACTION**, **BEGIN** o **BEGIN WORK**. Aunque algunos sistemas de gestión de bases de datos (como Oracle) permiten omitir este paso, es recomendable hacerlo.
2. Realizar las operaciones deseadas de **inserción**, **actualización** o **eliminación** de **filas** en la base de datos.
3. **Finalizar** la **transacción** con un **COMMIT** para confirmar los cambios de manera permanente o un **ROLLBACK** para deshacer los cambios en caso de existir algún problema.

En la siguiente figura 2, se muestra un ejemplo del uso de las sentencias START TRANSACTION y COMMIT en una tabla 'cliente' recién creada.

```
CREATE TABLE cliente (
    id_cliente INT PRIMARY KEY,
    nombre CHAR (20)
);

START TRANSACTION;
INSERT INTO cliente VALUES (1, 'Pepe');
COMMIT;
```

Fig.2. Ejemplo de transacción con COMMIT y START TRANSACTION sobre la tabla 'cliente' recién creada.

En el siguiente audio, exploraremos la variable **AUTOCOMMIT** y su impacto en la gestión de transacciones en bases de datos.



Audio 1. "Variable AUTOCOMMIT"
https://on.soundcloud.com/BZTDw8utxxGw6qoS7



/ 3. SAVEPOINT, ROLLBACK TO SAVEPOINT y RELEASE SAVEPOINT, y estado de datos durante la transacción

Durante el ciclo de vida de las transacciones se pueden utilizar otras sentencias, como son las de la figura 3, las cuales serán explicadas a continuación.

```
CREATE TABLE producto (
    id_producto INT PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    precio DOUBLE
);

INSERT INTO producto (id, nombre) VALUES (1, 'Primero');
INSERT INTO producto (id, nombre) VALUES (2, 'Segundo');
INSERT INTO producto (id, nombre) VALUES (3, 'Tercero');

-- Comprobamos las filas que existen en la tabla
SELECT *
FROM producto;

-- Ejecutamos una transacción que incluye un SAVEPOINT
START TRANSACTION;
INSERT INTO producto (id, nombre) VALUES (4, 'Cuarto');
SAVEPOINT sp1;
INSERT INTO producto (id, nombre) VALUES (5, 'Cinco');
INSERT INTO producto (id, nombre) VALUES (6, 'Seis');
ROLLBACK TO sp1;
```

Fig.3. Ejemplo de uso de SAVEPOINT y ROLLBACK TO SAVEPOINT.

3.1. SAVEPOINT

La instrucción SAVEPOINT permite crear puntos de guardado intermedios entre el inicio de la transacción y algún punto crítico de esta. Se les asigna un nombre, como se muestra en la anterior imagen. En caso de repetirse el nombre, se tiene en cuenta el más reciente.

Enlaces de interés...

En el siguiente enlace, encontrarás un vídeo que te ayudará a entender el uso y funcionamiento de estas instrucciones. Ten en cuenta que en Oracle no es necesario declarar explícitamente el inicio de la transacción con los comandos BEGIN o START TRANSACTION.

3.2. ROLLBACK TO SAVEPOINT

Permite deshacer la transacción hasta el estado de un determinado SAVEPOINT, anulando los cambios hechos desde que paso por él. La sintaxis es 'ROLLBACK TO nombreSavePoint', como se ve en la anterior imagen.

3.3. RELEASE SAVEPOINT

Esta instrucción elimina un SAVEPOINT indicando su nombre. Si el SAVEPOINT no existe, el sistema dará un error. Además, todos los SAVEPOINT de la transacción se eliminan al ejecutar un COMMIT o ROLLBACK. La sintaxis es 'RELEASE SAVEPOINT nombreSavePoint'.

3.4. Estado de los datos durante las transacciones

Escucha el siguiente audio para entender el estado en que se encuentran los datos durante el ciclo de vida de la transacción.

Audio 2. "Estado de los datos durante la transacción"

https://on.soundcloud.com/1VXPp3k5qnMiaZUG9

/ 4. Problemas de concurrencia de transacciones

En las bases de datos, al ser mayormente multiusuario, suelen darse casos en los que **varios usuarios inicien transacciones cuya ejecución coincide en el tiempo**. Esto puede provocar una serie de problemas que veremos a continuación.

Enlaces de interés...

En el siguiente enlace, encontrarás un vídeo con el que podrás ampliar tus conocimientos sobre el concepto de concurrencia y entender en qué consiste el concepto de paralelismo, que es un tipo específico de concurrencia: https://www.youtube.com/watch?v=kMr3mF71Kp4

- **Lectura sucia (*Dirty Read*)**: Este problema ocurre cuando una segunda transacción accede a los datos que están siendo modificados por otra transacción antes de que los cambios sean confirmados, por lo que serían datos que pueden no ser definitivos. Se muestra en la siguiente tabla:

Transacción 1	Transacción 2
Begin	Begin
	Update table set age = 14 where id = 1
Select age from table where id= 1	
Commit	Commit

Tabla 2. Transacciones con lectura sucia.

- **Lectura no repetible (*Non-Repeatable Read*)**: Este problema se presenta cuando una transacción consulta un dato en dos momentos distintos y descubre, al realizar la segunda consulta, que el dato ha sido cambiado o eliminado por otra transacción. Se muestra en la siguiente tabla:

Transacción 1	Transacción 2
Begin	Begin
Select age from table where id= 1	
	Update table set age = 14 where id = 1
	Commit
Select age from table where id = 1	
Commit	

Tabla 3. Lectura no repetible en transacciones.

- **Lectura fantasma (*Phantom Read*)**: El problema de la 'lectura fantasma' ocurre cuando, durante una transacción, se detectan nuevas filas en un conjunto de datos al repetir una consulta. Esto sucede porque otra transacción añadió filas en el intervalo de tiempo entre las consultas.

- **Problema de la modificación perdida**: Este problema ocurre cuando varias transacciones intentan modificar el valor de la misma fila basándose en su valor inicial. En este caso, la última modificación realizada sobrescribirá los cambios efectuados por las transacciones anteriores.

Vídeo 2. "Ejemplos de problemas de concurrencia"

https://bit.ly/3MZeugn

/ 5. Control de concurrencia: Técnicas de bloqueo pesimistas y optimistas

Para **controlar la concurrencia**, existen distintos **tipos de bloqueos** que la base de datos puede aplicar, según la tarea que se esté realizando.

Hay dos técnicas de bloqueo para prevenir los problemas de concurrencia en una base de datos. Sus características se detallan en la siguiente tabla:

Tipo de bloqueo	Descripción
Pesimista	Consiste en la **prevención del interbloqueo**. Se aplica inmediatamente después de solicitarlo, ya sea sobre una tabla o un registro. Esto asegura que la actualización de la información se realice con éxito, ya que previene que otros usuarios accedan a los datos mientras están bloqueados.
Optimista	Consiste en la **detección del interbloqueo**. La tabla o el registro se bloquea únicamente durante la actualización en el disco, y no durante la modificación de los datos en sí. Es ideal cuando es poco probable que otro usuario intente modificar los datos al mismo tiempo que se hacen cambios.

Tabla 4. Bloqueo pesimista y optimista.

Además de los bloqueos, existen otras técnicas para el control de la concurrencia, como son las **marcas de tiempo** y la **validación**.

Los **bloqueos** están **principalmente determinados por** si las **operaciones** que se desean ejecutar **simultáneamente** son de **lectura o escritura de datos**. En Oracle, hay un conjunto de **reglas establecidas** que regulan estos casos. Veremos estas reglas en la siguiente figura 4.

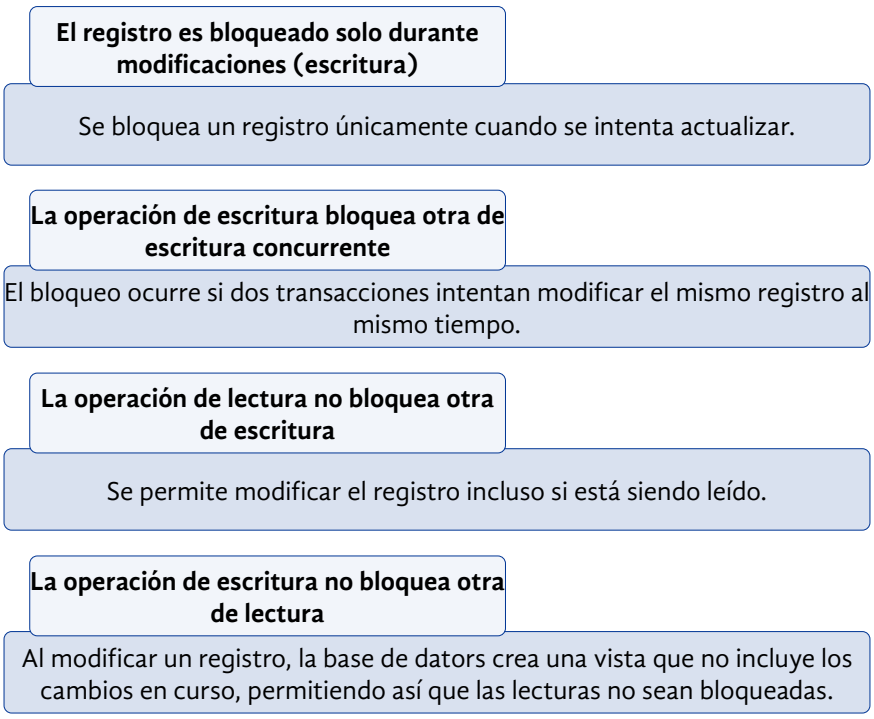


Fig.4. Reglas de bloqueo de Oracle.

A continuación podrás ver un audio sobre el punto anterior:

Audio 3. "Concurrencia en bases de datos"
https://on.soundcloud.com/jZgsWjveWcP8Rbhs9

/ 6. Transacciones distribuidas

Las transacciones distribuidas son aquellas que involucran múltiples nodos en una red, afectando diversos recursos. Estas transacciones requieren coordinación para confirmar (COMMIT) o deshacer (ROLLBACK) los cambios realizados en cada nodo.

6.1. Requerimientos de transacciones distribuidas

Para ejecutar una transacción distribuida, es esencial que haya una **comunicación efectiva entre los distintos nodos de la red y se garantice la atomicidad de la transacción**. Esto implica que:

- **Debe existir un 'agente raíz' que inicie la transacción**.
- El 'agente raíz' **debe asegurar que la transacción se inicie** (BEGIN-TRANSACTION) **y se complete** (COMMIT o ROLLBACK) para toda la transacción distribuida.

6.2. Recuperación de transacciones distribuidas

Cada nodo que participa en la transacción cuenta con un **Manejador de Transacciones Locales (LTM)**, mientras que un **Manejador de Transacciones Distribuidas (DTM)** coordina la transacción a nivel global. Los **LTM son responsables de ejecutar subtransacciones locales** que forman parte de la transacción distribuida. **El DTM asegura que**:

- Se mantenga la **atomicidad a nivel local**.
- Se ejecuten **todas o ninguna de las acciones** de la transacción distribuida en cada nodo.
- Todos los nodos lleguen a la **misma decisión global** (COMMIT o ROLLBACK) para mantener la **consistencia de los datos**.



Fig.5. Transacciones distribuidas y subtransacciones.

6.3. COMMIT de dos fases

El agente raíz actúa como coordinador y es responsable de decidir si la transacción global se confirmará (COMMIT) o se deshará (ROLLBACK). Este proceso se realiza en dos fases:

- **Primera fase**: Determinación de una decisión global común.
- **Segunda fase**: Ejecución de la decisión tomada por todos los nodos a nivel local.

Este mecanismo asegura que todas las operaciones de la transacción distribuida se realicen de manera coherente en todos los nodos involucrados.

/ 7. Políticas de bloqueo

Cuando se trabaja con bases de datos de manera simultánea, es necesario establecer reglas y estrategias para que un sistema de gestión de bases de datos (DBMS) gestione los bloqueos durante las transacciones. Estos bloqueos regulan el acceso concurrente a los datos para asegurar su integridad y consistencia.

Las políticas de bloqueo **determinan cuándo aplicar y liberar bloqueos en recursos como tablas, filas o páginas**. Son esenciales para **evitar problemas de concurrencia, como inconsistencias de datos y bloqueos mutuos (*deadlocks*)**.

A continuación, veremos los componentes clave de una buena política de bloqueos.

7.1. Niveles de bloqueo

Lo primero que debemos entender es qué objeto de la base de datos se va a bloquear. **Los SGBD pueden establecer bloqueos en diferentes niveles** dependiendo de lo que estimen más oportuno.

- **Nivel de tabla**: Bloquea toda la tabla, evitando que otras transacciones lean o modifiquen cualquier parte de ella. En la siguiente figura 6 se muestra un ejemplo.

```
BEGIN
    -- Establecer un bloqueo exclusivo en la tabla 'pedidos'
    LOCK TABLE pedidos IN EXCLUSIVE MODE;
    -- Realizar operaciones de escritura en la tabla
    UPDATE pedidos SET estado = 'procesado' WHERE id = 123;
    -- Confirmar la transacción y liberar el bloqueo
    COMMIT;
END;
```

Fig.6. Bloqueo de tabla.

- **Nivel de fila**: Realiza el bloqueo de filas individuales de la tabla. Dado que no bloquea la tabla entera, permite una mayor concurrencia, ya que dos transacciones pueden modificar concurrentemente. En la siguiente figura 7 se muestra un ejemplo.

```
BEGIN
    -- Bloquear la fila del pedido con pedido_id = 456
    SELECT 1 FROM pedidos WHERE id = 456 FOR UPDATE;
    -- Actualizar el estado del pedido
    UPDATE pedidos SET estado = 'procesado' WHERE id = 456;
    -- Confirmar la transacción y liberar el bloqueo
    COMMIT;
END;
```

Fig.7. Bloqueo de fila.

- **Nivel de página**: Realiza el bloqueo de una página de datos que puede contener varias filas.

7.2. Tipos de bloqueos

Una vez identificados los elementos de la base de datos que pueden ser bloqueados, es esencial determinar la **granularidad del bloqueo** a aplicar. Esto implica **elegir el tipo adecuado según la necesidad de proteger la consistencia de los datos y permitir el acceso concurrente**.

Es importante equilibrar la seguridad y el rendimiento para mantener la integridad de los datos sin comprometer la eficiencia del sistema. Los principales tipos de bloqueo que se pueden aplicar sobre los datos u objetos de la base de datos son:

- **Bloqueo compartido (*Shared Lock*, S)**: Permite que múltiples transacciones lean el mismo recurso, pero ninguna tiene opción de modificarlo mientras el bloqueo esté en vigor. Oracle no admite este tipo de bloqueos, pero sí otros SGBD. En la siguiente figura 8 se muestra un ejemplo de cómo se realizaría en MySQL.

```
START TRANSACTION;
-- Leer el stock disponible con un bloqueo compartido
SELECT stock FROM productos WHERE id = 123 LOCK IN SHARE MODE;
-- Confirmar la transacción sin modificación
COMMIT;
```

Fig.8. Sentencia SQL de una transacción en la que se utiliza bloqueo compartido en MySQL.

- **Bloqueo exclusivo (*Exclusive Lock*, X)**: Permite que una sola transacción modifique el recurso, bloqueando tanto las lecturas como las escrituras por parte de otras transacciones. En la siguiente figura 9 se muestra un ejemplo de cómo se realizaría en Oracle.

```
BEGIN;
-- Bloquear la fila para modificación
SELECT estado FROM pedidos WHERE id = 123 FOR UPDATE;
-- Modificar el campo estado
UPDATE pedidos SET estado = 'procesado' WHERE id = 123;
-- Confirmar la transacción y liberar el bloqueo
COMMIT;
```

Fig.9. Bloqueo exclusivo en Oracle.

- **Bloqueo de actualización (*Update Lock*, U)**: Es un tipo de bloqueo que se utiliza en algunas bases de datos para prevenir que ocurran bloqueos mutuos durante las operaciones de actualización. **Realiza el bloqueo de escritura** para evitar que otras transacciones modifiquen el dato, **pero permite las lecturas** mientras tanto.

Enlaces de interés...

En el siguiente enlace, encontrarás la documentación oficial de Oracle en la que podrás descubrir la sintaxis del comando Lock y todas las opciones que ofrece para su utilización: https://docs.oracle.com/cd/B19306_01/server.102/b14200/statements_9015.htm

/ 8. Niveles de procesamiento de consulta: Procesadores locales y procesadores distribuidos

En los sistemas de bases de datos distribuidos, las consultas pueden ser procesadas a través de dos niveles distintos: los **procesadores locales** y los **procesadores distribuidos**.

8.1. Procesadores locales

Los procesadores locales **gestionan las consultas y operaciones en una base de datos específica en un solo nodo o servidor**. En un sistema distribuido, cada base de datos puede tener su propio procesador local para realizar estas tareas:

- **Interpretar y ejecutar consultas**: Convierte consultas SQL de alto nivel en operaciones de bajo nivel sobre datos locales.
- **Optimización local**: Mejora la eficiencia de las consultas a nivel local mediante técnicas como la **selección de índices adecuados y la planificación de ejecución**.
- **Manejo de transacciones y bloqueos**: Gestiona la concurrencia y asegura la integridad de los datos dentro de la base de datos local.

8.2. Procesadores distribuidos

Los procesadores distribuidos se encargan de coordinar la ejecución de consultas y transacciones que abarcan múltiples bases de datos ubicadas en diferentes nodos. Su función incluye:

- **Descomposición de consultas**: Dividen las consultas globales en subconsultas ejecutables localmente en diferentes nodos.
- **Optimización global**: Planifican la mejor manera de ejecutar las subconsultas distribuidas, **minimizando la comunicación entre nodos y el tiempo total de ejecución**.
- **Recolección y combinación de resultados**: Tras procesar los datos localmente, el procesador distribuido combina los resultados para generar la respuesta final.
- **Manejo de transacciones distribuidas**: Garantiza la consistencia de los datos y el compromiso de transacciones en varias bases de datos. **Utilizan el 'commit' de dos fases**.



Fig.10. BD distribuidas.

Estos niveles permiten a las bases de datos distribuidas gestionar de forma eficiente consultas y manipulaciones de datos en varias ubicaciones, mejorando la escalabilidad y la resiliencia del sistema.

/ 9. Caso práctico 1: “Manejo de transacciones”

Planteamiento: En este caso práctico, aplicaremos los conceptos teóricos aprendidos sobre transacciones en bases de datos. Nos enfocaremos en el uso de las sentencias **COMMIT**, **SAVEPOINT** y **ROLLBACK**.

Nudo: Propondremos un escenario práctico en el que se llevará a cabo una transacción. Para ello, crearemos una tabla de prueba con dos columnas y agregaremos algunos datos. Luego, realizaremos una transacción para modificar el valor de la columna 1 si la columna 2 tiene un valor específico, por ejemplo, 20. En este proceso, utilizaremos un **SAVEPOINT** para marcar un punto de restauración intermedio.

¿Es posible deshacer los cambios una vez que se ha ejecutado la sentencia **COMMIT** utilizando **ROLLBACK**?

Desenlace:

```
CREATE TABLE prueba(
    Dato1 NUMBER,
    Dato2 NUMBER
);

INSERT INTO prueba VALUES (5, 20);
SELECT * FROM prueba;
COMMIT;

SET TRANSACTION NAME 'transaccion_prueba';
UPDATE prueba SET Dato1 = 2 WHERE Dato2 = 20;
SAVEPOINT puntoSalvado;
UPDATE prueba SET Dato2 = -1 WHERE Dato1 = 2;
ROLLBACK TO puntoSalvado;
```

Fig.11. Ejemplo de composición interna sobre las tablas indicadas.

En la figura 11, se muestra una posible solución al problema planteado.

Una vez ejecutada la sentencia **COMMIT**, los cambios realizados se hacen permanentes en la base de datos, y no es posible deshacer estos cambios con un **ROLLBACK**.

/ 10. Caso práctico 2: "Padres e hijos"

Planteamiento: Un cliente ha solicitado nuestra ayuda para realizar una operación en la tabla 'hijo', en la cual necesita insertar una nueva fila. El cliente quiere asegurarse de que este nuevo registro esté adecuadamente vinculado a un registro existente en la tabla 'padre'. Es importante verificar esta relación para mantener la consistencia de los datos entre ambas tablas y cumplir con los requisitos del cliente.

Id_padre	Nombre

Tabla 5. Columnas de la tabla 'padre'.

Id_hijo	Nombre	Id_padre

Tabla 6. Columnas de la tabla 'hijo'.

Nudo: Quiere garantizar que, al insertar un nuevo registro en la tabla 'hijo', no se haya eliminado el padre correspondiente de la tabla 'padre'. ¿Cómo puede asegurarse de que esto ocurra?

Desenlace: Para asegurarse de que el padre asociado no sea eliminado mientras se inserta un nuevo registro en la tabla 'hijo', es necesario implementar un bloqueo de lectura en la tabla 'padre'. Esto se puede lograr con la siguiente instrucción:

```
SELECT * FROM padre
WHERE nombre = 'Diego Rodríguez'
FOR UPDATE;
```

Fig.12. Bloqueo de fila.

Al utilizar esta sentencia, la fila correspondiente al padre indicado quedará protegida contra la eliminación por parte de otros usuarios, ya que el bloqueo compartido evita que se realicen cambios o eliminaciones en esa fila mientras el bloqueo esté en vigor. De esta manera, se garantiza que el registro del padre no será eliminado durante la operación de inserción en la tabla 'hijo'.

/ 11. Resumen y resolución del caso práctico de la unidad

En este tema, hemos analizado el concepto de transacciones, abarcando sus propiedades, estados, comandos y métodos de control. También hemos tratado la concurrencia, los problemas que puede ocasionar y las estrategias para manejarla adecuadamente. Además, hemos explorado las transacciones distribuidas y los niveles de procesamiento de una consulta.

Finalmente, hemos examinado las políticas y los tipos de bloqueo que se utilizan en los sistemas de bases de datos.

A continuación podrás ver en la siguiente página el esquema resumen del tema:

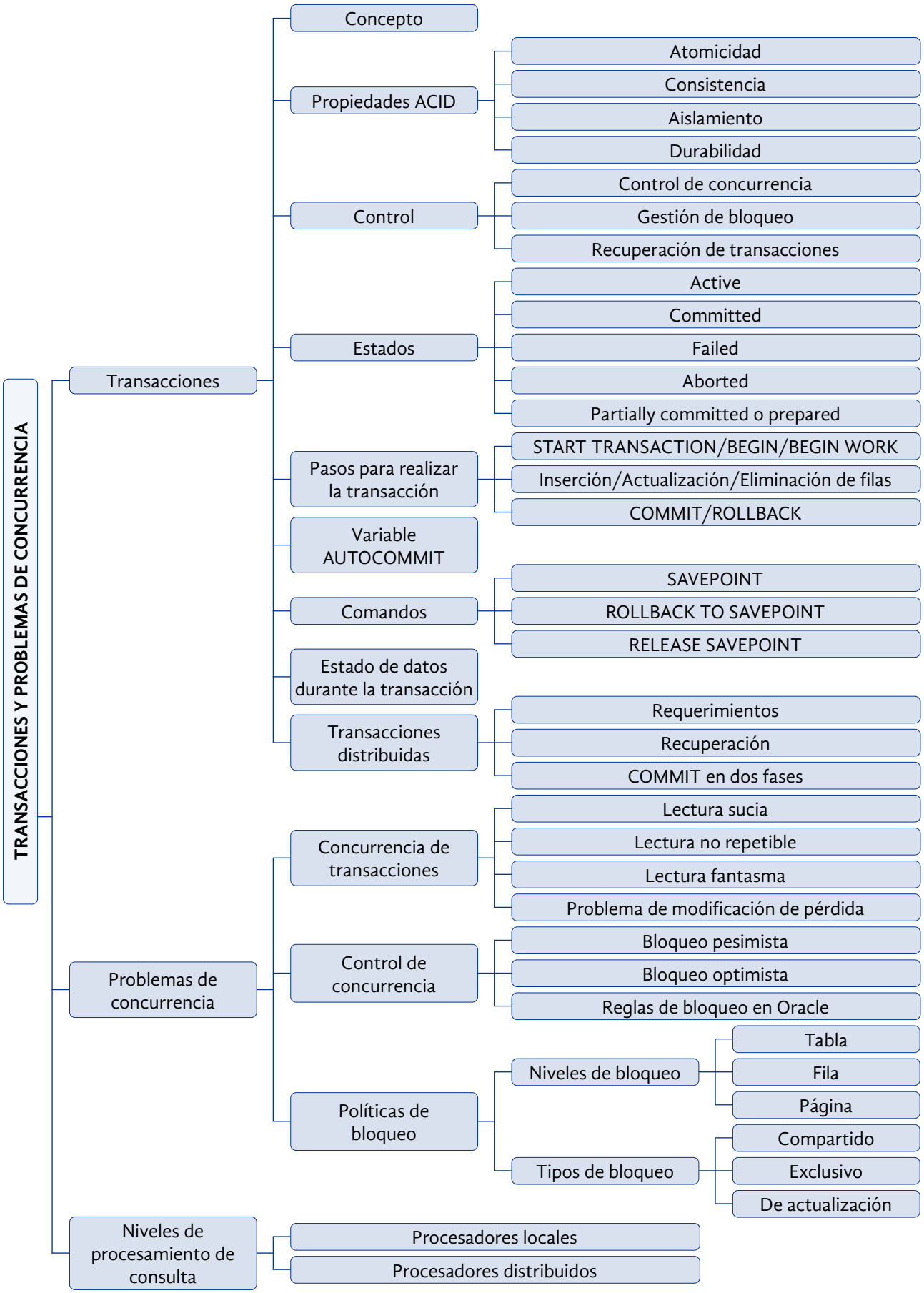


Fig.13. Esquema resumen del tema.

Resolución del caso práctico de la unidad

Para las situaciones descritas, como la entrada simultánea de mercancías o la modificación de salarios, es fundamental usar transacciones y políticas de bloqueo para garantizar la consistencia de los datos y evitar errores. Las transacciones aseguran que las operaciones se completen de forma exitosa o se reviertan completamente, manteniendo la integridad de los datos. Los bloqueos previenen que múltiples usuarios modifiquen el mismo dato al mismo tiempo, asegurando actualizaciones seguras.

Estos mecanismos de control de concurrencia son esenciales en bases de datos multiusuario, evitando problemas como inconsistencias y los problemas de concurrencia que hemos visto en este tema, y garantizando operaciones coherentes y fiables.



Fig.14. Las políticas de bloqueo garantizan la consistencia de la información.

/ 12. Bibliografía

Connolly, T., y Begg, C. (2005). *Sistemas de Bases de Datos (5ª ed.)*. Madrid, España. Addison Wesley.

Elmasri, R., & Navathe, S. (2007). *Fundamentos de sistemas de bases de datos*. Addison Wesley.

López Montalbán, I., & Castellano Pérez, M. J. (2014). *Gestión de bases de datos: Técnico Superior en Administración de Sistemas Informáticos en Red*. Madrid. Garceta D. L.

Oppel, A. (2009). *Databases A Beginner's Guide*. McGraw-Hill Education.

Pérez Marqués, M. (2016). *Administración básica de bases de datos con Oracle 12c SQL*. Madrid, España: Alfaomega.

Sánchez, G. C. (2001). *Sistemas gestores de bases de datos*. Paraninfo.